JHQ-GPU: Representation-Driven GPU Execution and Workload-Adaptive Hierarchical Refinement

Rui Luo and
Mengxuan Zhang[0000-0001-8155-4942]

Australian National University, Canberra, Australia
u8076655@anu.edu.au, mengxuan.zhang@anu.edu.au

Abstract. JHQ separates approximate nearest-neighbour scoring into a cheap primary stage and selective residual refinement. A direct GPU port, however, exposes a design tension rather than a single bottleneck: the primary stage scores many candidates with a compact code, so lookup-table residency and instruction issue matter; the residual stage touches far richer per-dimension state, so unnecessary refinement is expensive. We therefore derive the GPU design from JHQ's representation instead of stacking generic CUDA optimisations. The Cartesian structure inside each existing 8-bit primary code lets a 256-entry query table be evaluated exactly as two 16-entry tables, occupying the middle ground between a full materialised LUT and table-free symbolic evaluation. Subspace-major layout follows from the warp's access pattern, while 32-bit packing reduces issued load, address and loop instructions without claiming fewer logical bytes. The remaining algorithmic question is how many survivors to refine; we select this budget with a label-free output-stability rule calibrated on workload queries. On six datasets from 768 to 3,072 dimensions and up to 17.8M vectors, mechanism controls indicate an issue-sensitive primary scan on the tested RTX 5090: table-free exact evaluation loses 30–52%, packing gains 30–48% despite moving the same logical bytes, and four- and eight-way factorisations that store byte-identical tables differ by up to 38% on lookup count alone. Factorisation grows more valuable as the number of subspaces and the probe depth rise, because a table that misses the on-chip budget is paid once per candidate per subspace. Against a matched, thread-pinned CPU baseline swept over its own refinement envelope, the same refinement knob is worth 2.5–3× more on the GPU than on the CPU. End-to-end, the competitive frontier changes with recall and batch size, yielding a bounded but explainable operating region for GPU JHQ rather than a universal-fastest claim.

Keywords: Approximate nearest neighbour search · GPU · Quantisation · High-dimensional data · Adaptive query processing

1 Introduction

High-dimensional approximate nearest-neighbour (ANN) search is a core primitive in retrieval, recommendation, and vector databases. On a GPU, compressed search is not governed by memory footprint alone. A representation can reduce bytes yet increase lookup pressure, instruction issue, or irregular access; conversely, an implementation can move the same logical bytes and still accelerate by issuing fewer instructions. The systems problem is therefore to map the algorithm's structure to the hardware bottleneck, not to assume that every reduction in storage or every wider load is automatically beneficial.

JHQ [3] is a useful case study because its hierarchy exposes two qualitatively different costs. After IVF routing, a compact primary code scores many candidates, whereas a per-dimension residual code is read only for the c_k survivors of primary filtering. A naive port can mishandle both sides: generic candidate-major codes give the warp a strided primary access pattern, conventional 8-bit query tables grow with the number of subspaces, and an over-large c_k converts a selective residual stage into unnecessary work. JHQ also leaves the refinement factor α in $c_k = \alpha k$ externally specified. Thus the GPU problem has two coupled performance questions with two different causes: how should the representation execute, and how much of the expensive hierarchy should execute?

Our premise is that a GPU design should follow from explicit constraints and rejected alternatives. We preserve the represented JHQ distances and ask, for each hot-path decision, what resource is under pressure, what representation property can relieve it, and what the alternative would cost. This separates research claims from engineering controls: a transpose is useful because the scan is candidate-parallel at fixed subspace, not because transposes are novel; packing is useful because it removes issued work, not because it magically moves fewer bytes; and a smaller LUT is useful only until extra lookups cost more than additional residency saves.

The strongest representation-specific result comes from this last trade-off. A conventional 8-bit ADC table materialises 256 distances per subspace. JHQ's primary codebook is Cartesian, so squared Euclidean distance separates across its one-dimensional levels. The same represented distance can therefore be computed from two 16-entry tables. This is not simply "smaller is better": the full table minimises lookups but competes for the same on-chip budget as the scan's own candidate scratch, while a table-free formulation minimises table storage but adds arithmetic and issued instructions. The two-table factorisation is a middle point in this computation–residency design space. Mechanistically, full-table state grows as $256M$ entries while the two-way factorised state grows as $32M$, so the factorised representation approaches any fixed on-chip capacity limit with an $8\times$ smaller coefficient. Experiments confirm the prediction: the factorisation matters much more at M=384 than at M=96, whereas still-smaller four- and eight-group tables are consistently slower.

The second contribution addresses a different decision: residual work. Refining every routed candidate would erase the hierarchy's main advantage, while a fixed small α can discard neighbours before the accurate stage sees them.

We therefore treat c_k as a workload-level control variable. Our output-stability criterion compares the returned top-k set at a smaller budget with a generous $\alpha_{\max}$ reference on sampled workload queries. We choose this signal because it is available at deployment time without labelled nearest neighbours or a learned query-difficulty model; the trade-off is that it is a proxy, not a recall certificate, and its sample size must be chosen for the workload. A matched CPU baseline shows this knob is not a portable one: pinning α=100 costs the CPU 8.7–41.8% of its throughput and the GPU 21.8–112.5% on the same dataset and the same recall targets, so the control JHQ leaves external is worth 2.5 to 3× more on the device we port it to.

Our evaluation uses six datasets from 768 to 3,072 dimensions, up to 17.8M vectors, on one RTX 5090. We use positive and negative controls to test the design hypotheses rather than report only the winning implementation. Table-free distance reduces table state yet loses 30–52%; four- and eight-way factorisations are byte-identical to each other yet the eight-way form is 5.8–38.3% slower; packed loading moves the same logical bytes yet gains 30–48%; a probe-cursor correction reduces issued loop work and can have the largest speedup despite zero algorithmic novelty. These results jointly indicate an issue-sensitive primary scan whose optimal design is not predicted by byte counting alone. End-to-end, JHQ leads IVF-RaBitQ through Recall@10=0.95 at large batch on four directly comparable datasets but yields in parts of the high-recall tail, and batch size can change the relative winner.

Contributions. First, we derive and evaluate a representation-driven GPU primary path for JHQ: access-pattern-driven layout, representation-aligned packed loads, and an exact Cartesian LUT factorisation whose two-way split is justified against full-table, finer-factorisation, and table-free alternatives. The JHQ-specific research claim is the factorisation and its hardware regime; standard layout and instruction-level changes are supporting engineering. Second, we propose a label-free, predictor-free workload policy for selecting JHQ's residual refinement budget, quantify its sample risk and amortisation, and show on a matched CPU baseline that the payoff of this control is several times larger on the GPU than on the CPU. Third, we evaluate the system with mechanism-oriented ablations, negative results, recall–QPS frontiers, batch sensitivity, and build/memory costs, explicitly reporting where competing GPU methods are stronger and where current evidence does not generalise.

2 Preliminaries

JHQ represents each database vector using a square orthogonal transform, a Cartesian primary codebook, and a per-dimension residual code [3]. This section introduces only the properties needed later; it does not repeat JHQ's training derivation.

Let $x \in \mathbb{R}^d$ be a database vector and $q \in \mathbb{R}^d$ a query. JHQ applies an orthogonal matrix $Q \in \mathbb{R}^{d\times d}$: $y = Qx$ and $q' = Qq$. Q is square; the transform is not dimensionality reduction. For a row-major batch X, the same convention is

$Y = XQ^\top$. Orthogonality preserves Euclidean distance, so the transform changes the coordinate system rather than the neighbour ordering.

The transformed vector is partitioned into M disjoint subspaces of dimension D_s, with $d = MD_s$. Each primary subspace code has B bits and selects one of $K = 2^B$ codewords. The important property is how those codewords are constructed: each is a Cartesian product of D_s one-dimensional Lloyd–Max levels. If L is the number of levels per coordinate, then $L = 2^{B/D_s}$. Hence the implementation admits only D_s dividing B. With B=8, D_s is in $\{1, 2, 4, 8\}$, corresponding to $\{8, 4, 2, 1\}$ primary bits per coordinate. The experiments in this paper use the D_s=8 regime, so each coordinate contributes one primary bit and each subspace is represented by one byte.

Let $\hat{y}_{\text{primary}}$ be the reconstruction from the primary codes. JHQ defines the residual as $r = y - \hat{y}_{\text{primary}}$. This residual is not the displacement from an IVF centroid. IVF is an external routing layer that restricts the candidate set; it does not redefine what the residual quantiser represents. The residual code is per dimension rather than per subspace. With B_r residual bits per dimension, the storage is approximately dB_r bits per vector. At B_r=8 and D_s=8, the logical JHQ representation is therefore about 1 primary bit plus 8 residual bits per dimension, before routing metadata and implementation overhead.

For a query, IVF routing yields a candidate set $C(q)$. The primary score D_P is evaluated for every routed candidate. For exposition we write $c_k = \alpha k$ as the budget relation; the CUDA implementation uses the integer form $c_k = \max(k, \lceil \alpha k \rceil)$. All evaluated α values are at least one, so the max only enforces the natural k-survivor floor. JHQ retains the c_k candidates with the smallest primary scores and evaluates the hierarchical residual score D_H only for those survivors. The final answer is the top-k under D_H among the survivor set. Thus α controls an irreversible filtering point: a neighbour removed by primary ranking cannot be recovered by residual refinement. Routing can independently remove true neighbours before the primary stage. These two loss sources must remain conceptually separate throughout the paper.

3 GPU-Native JHQ

3.1 Design from the cost structure

We do not begin with CUDA primitives; we begin with the work JHQ requires after routing. Let n_c be the routed candidate count. The primary stage performs work proportional to $n_c M$ over compact codes, while the residual stage performs work proportional to $c_k d$ over richer per-dimension codes, with $c_k = \alpha k$ and typically $c_k \ll n_c$. This asymmetry yields three design requirements: (i) make the primary code access match candidate-parallel warp execution; (ii) keep the primary distance representation resident without replacing JHQ's quantiser; and (iii) preserve selective residual access rather than fusing accurate scoring into the broad scan. Figure 1 shows the resulting pipeline. The semantic invariants are fixed: the orthogonal transform, primary codebook, residual definition, and

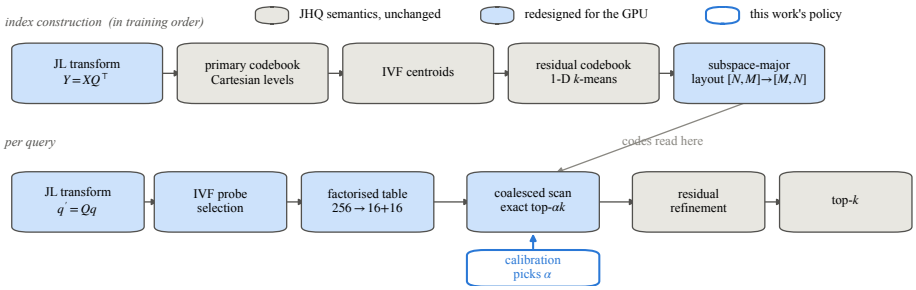

Fig. 1: JHQ build and query pipeline. GPU redesign changes execution and physical layout while preserving the original hierarchy. The scan uses bounded per-thread top-αk selection; Proposition 1 characterises the ideal exact selector, while Section 4.3 quantifies the implementation's measured deviation.

hierarchical ordering remain JHQ's; only physical representation, execution, and the external refinement policy change.

3.2 Access pattern determines physical layout

Logically the primary codes form an $N \times M$ matrix. During the broad scan, warp lanes score different candidates while advancing through the same subspace m. Under candidate-major $[N, M]$ storage, the warp therefore reads one byte per lane with stride M; the layout is convenient for reconstructing one vector but mismatched to the hot access pattern. Storing the scan copy in subspace-major $[M, N]$ order makes those 32 candidate bytes adjacent. The reason for the transpose is thus specific and falsifiable: it aligns the contiguous memory dimension with the dimension shared by the warp. Transposition itself is standard GPU engineering and is not a novelty claim.

The $D_s \mid B$ representation constraint creates a separate instruction-level opportunity. In the paper setting $D_s{=}B{=}8$, one subspace code is one byte, so four subspace codes form a natural 32-bit unit. A packed scan can load that word once and unpack in registers. This is not another coalescing argument: after the layout fix, the required sectors are already appropriate, and packing does not reduce the logical code bytes. It reduces load instructions, address calculations, and loop iterations. We choose 32-bit packing because it is the first native word width exposed directly by four one-byte subspaces; we do not claim that ever-wider packing is universally better, since wider words can add unpacking, dependencies, alignment constraints, and register pressure. That unmeasured design space is deliberately outside the claim.

3.3 The primary-distance design space

The central question is not "how small can the LUT be?" but "how much table state should be materialised before saved residency is outweighed by additional issued work?" Three points define the space: a full table minimises per-candidate lookup count but maximises resident state; a table-free symbolic form minimises

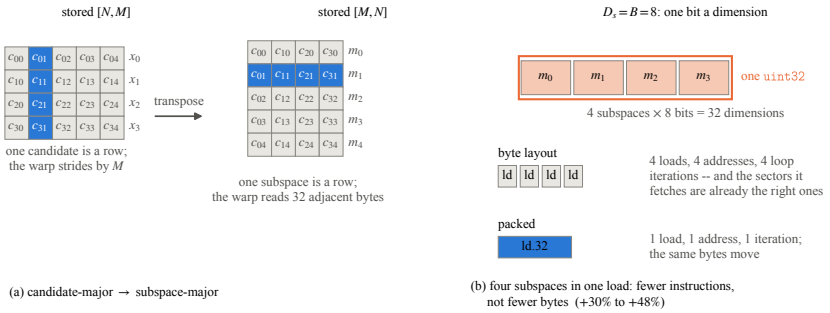

Fig. 2: Logical candidate-major versus GPU scan layout, and the separate role of packed 32-bit access.

table state but adds arithmetic; and Cartesian factorisation trades one extra lookup for a large reduction in resident entries while preserving the represented distance. We use the latter only if its representation-specific algebra supports it.

Cartesian-factorised primary distance. For subspace m and code c, let $T_m[c] = \|q'_m - C_m[c]\|_2^2$. A conventional 8-bit implementation materialises 256 values per subspace. Because JHQ constructs $C_m[c]$ as a Cartesian product of one-dimensional levels and squared Euclidean distance is additive, any partition of the eight base-2 code decisions induces an exact sum of smaller tables. For the two-way partition used here, $c_{\mathrm{hi}} = c \gg 4$ and $c_{\mathrm{lo}} = c \bmod 16$, giving

$$T_m[c] \;=\; T_m^{\mathrm{hi}}[c_{\mathrm{hi}}] \;+\; T_m^{\mathrm{lo}}[c_{\mathrm{lo}}]. \tag{1}$$

Each component table has 16 entries, so G=2 stores 32 entries per subspace and performs two lookups. The reduction in table-construction work is 16×, not merely the 8× reduction in resident entries: the full table evaluates $256D_s$ coordinate contributions per subspace, whereas the split form evaluates $32(D_s/2) = 16D_s$ contributions. More generally, an equal G-way partition stores $G \cdot 2^{8/G}$ entries but requires G lookups per candidate–subspace. This exposes the trade-off analytically: increasing G lowers resident table state only until the table fits the target on-chip regime, while instruction demand continues to rise.

The residency target is narrower than the device's shared-memory limit, because the table is not the only claimant. The scan kernel's own admission test is

$$\texttt{scan\_base} + M \cdot G \cdot 2^{8/G} \cdot 4 \;\le\; \texttt{smem\_optin}, \qquad \texttt{scan\_base} = 40 \cdot \texttt{BLOCK} \text{ bytes}, \tag{2}$$

where `scan_base` is the per-thread candidate scratch claimed before the table. On the tested card `smem_optin` = 101,376 B, so the budget actually left for the table is 94.0 KiB at `BLOCK`=128 and falls to 59.0 KiB at `BLOCK`=1024. The full G=1 table is $M \cdot 256 \cdot 4$ bytes: 98,304 B = 96 KiB at M=96 and 393,216 B = 384 KiB at M=384. *Neither* clears the budget at any admissible block size — M=96 misses the most generous of them by 2 KiB — whereas every factorised

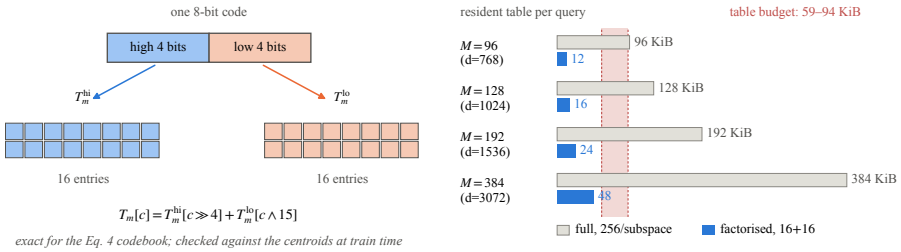

Fig. 3: Exact Cartesian factorisation of an 8-bit JHQ primary lookup table: 256 resident entries become two 16-entry tables. The shaded band on the right is the budget Equation 2 leaves for the table across the admissible block sizes. Every full table falls outside it and every factorised table inside.

table clears the least generous, at 12 KiB and 48 KiB respectively (Figure 3). The full table is therefore off chip at both M, and what grows with M is not whether the table is resident but the cost of the lookups that miss: the scan performs one lookup per candidate per subspace, so a non-resident table is paid M times per candidate. G=2 is thus a hypothesis about staying inside a residency budget that the full table never meets, at the smallest added lookup count — not an arbitrary split.

Section 6.4 tests both sides of that hypothesis. G=2 is at least as fast as G=1, G=4 and G=8 in all eight reported configurations, with one exact tie against G=1. At M=96 the factorisation is worth 0–11%; at M=384 it is worth 8–53%, rising monotonically with probe depth in both cases. Crucially, G=4 and G=8 store *byte-identical* tables — $G \cdot 2^{8/G}$ is 16 for both — and differ only in performing four versus eight lookups per candidate–subspace; the eight-way form is 5.8–38.3% slower. That pair is the one control in this paper where table residency is held exactly fixed while issued work doubles. A table-free exact formulation goes further in the "less state" direction and still loses 30–52%. Thus the objective is not minimum bytes or maximum occupancy in isolation. The chosen point balances residency against issued lookup and arithmetic work. Stage timing also shows LUT construction is only 0.2–2.3% of a batch, so the 16× reduction in table-construction work cannot explain the end-to-end gain by itself.

This contribution must also be separated from generic small-table ADC. PQ Fast Scan [1] and Quicker ADC [2] already show that lookup-table organisation is central to quantised search. The distinction is not that prior small-table methods are lossy — it is where the small table comes from. Quicker ADC obtains one by *changing the quantiser*, using narrower irregular sub-quantisers whose tables are small by construction. We change nothing: JHQ's existing 8-bit Cartesian codebook already satisfies Equation 1 exactly, so the same represented distance is recovered from 32 entries instead of 256 without touching a codeword. The research question is therefore representation-to-hardware mapping: when does this exact reorganisation move JHQ into a better residency/issue regime? Algebraic exactness is in real arithmetic and does not promise bitwise-identical floating-point association or tie order.

Instruction-aware primary scan. After routing, $D_P(i) = \sum_m T_m[c_{i,m}]$, or the equivalent sum of factorised components. Subspace-major storage addresses memory transaction shape; packed loading addresses instruction count. The distinction matters because the latter can improve throughput even when the same logical bytes move. On openai3-3072, factorisation gains remain similar on byte and packed layouts, so the two changes are complementary rather than one merely repairing the other. This is also why we keep packing out of the novelty claim: its scientific role is as evidence that the primary kernel is sensitive to issue work, while the JHQ-specific contribution is the exact factorisation that changes the table-residency regime.

Preserve selective residual refinement. The residual code is per dimension and therefore much richer than the one-byte primary subspace code. Reading it for every routed candidate would turn an $O(c_k d)$ selective stage into work closer to $O(n_c d)$, discarding the hierarchy's central systems advantage. We therefore keep primary screening and residual refinement separate: only the c_k primary survivors trigger residual accesses. A fused design is still a plausible alternative because it removes the intermediate top-c_k write/read round trip and one kernel boundary. The frozen negative control shows why we reject it: on stella-trec24 it is 14.2% slower than the 68,565-QPS baseline at *nprobe*=128 and 14.8% slower than the 16,126-QPS baseline at *nprobe*=512, because the fused kernel combines the shared-memory and register footprints of both phases and remains resident for the full scan. Larger α reduces only primary-filtering risk and increases this expensive residual work; it cannot recover neighbours lost during IVF routing.

3.4 What the execution design deliberately leaves unresolved

The representation tells us how to execute each unit of JHQ work cheaply, but it does not tell us how many expensive residual units are useful. The interface still ends at α, which scales the survivor budget c_k approximately as αk (with integer rounding and the k floor in the implementation). A global α large enough for a difficult workload can waste most of the residual stage on an easy one, while a small value can irreversibly filter useful candidates. Section 4 therefore treats α as a workload control problem rather than another kernel constant.

4 Adaptive Hierarchical Refinement

4.1 Why α needs workload-level control

Original JHQ leaves α externally specified. At *nprobe*=128, openai3-3072 is already flat from α=4 over the measured grid, whereas arxiv-768 is still improving at α=200. The useful budget therefore spans 4 to at least 200 in our measurements; the upper endpoint is censored rather than a plateau. A single global constant has an unavoidable failure mode: choose it high and easy workloads pay unnecessary $O(\alpha k d)$ residual work; choose it low and difficult workloads

lose candidates before accurate refinement. We deliberately calibrate a workload-level operating point rather than predict α per query: this keeps the control path label-free and amortisable, at the cost of assuming a reasonably stable query distribution.

4.2 Output-stability criterion

Let Q_s be S sampled queries from the current workload and let $R_{\max}(q)$ be the returned top-k ID set under a generous $\alpha_{\max}$. For another tested α, let $R_\alpha(q)$ denote the corresponding top-k set. We define the total slot disagreement

$$D(\alpha) \;=\; \sum_{q \in Q_s} \big[\, k - |R_\alpha(q) \cap R_{\max}(q)| \,\big]. \tag{3}$$

The tolerance ϵ is a non-negative integer over the entire sample, not a per-query fraction or a positional rank mismatch. We select the smallest tested-grid α satisfying $D(\alpha) \le \epsilon$. The design choice is intentional: set membership asks whether additional residual work changes which neighbours are returned, while ignoring harmless permutations of the same IDs. The current S=32 and ϵ=1 are empirical defaults, not universal constants or correctness guarantees.

Why use output stability instead of recall, a learned predictor, or a fixed heuristic? Ground-truth recall is the quantity we ultimately care about but is usually unavailable online; a learned predictor can adapt per query but introduces labelled training, model maintenance, and drift behaviour outside this paper's scope; a fixed heuristic cannot observe workload difficulty at all. Output stability is available from the running index itself and directly measures the marginal effect of spending more residual work. Its limitation is equally important: agreement with $\alpha_{\max}$ cannot recover IVF routing losses, cannot certify exact recall, and cannot reveal improvements beyond $\alpha_{\max}$. We therefore validate the proxy against ground truth offline rather than treating self-agreement as a guarantee.

4.3 Monotonicity and bisection

We first evaluate $\alpha_{\max}$ to construct the reference, then search a discrete α grid by bisection rather than evaluate every value. The efficiency argument requires a monotone acceptance predicate, so we state the condition explicitly.

Proposition 1 (monotonicity of disagreement). *Let $\alpha_1 \le \alpha_2 \le \alpha_{\max}$. Under exact top-$c_k$ primary selection with c_k non-decreasing in α, a fixed refined distance, and a deterministic total tie rule, $D(\alpha_1) \ge D(\alpha_2) \ge D(\alpha_{\max}) = 0$.*

Proof. Let S_α be the primary-survivor set. Exact top-c_k selection gives $S_{\alpha_1} \subseteq S_{\alpha_2} \subseteq S_{\alpha_{\max}}$. Order $S_{\alpha_{\max}}$ by the fixed refined distance together with the deterministic tie key, and let $R_{\max}$ be its first k elements. Any element of $R_{\max}$ that is present in S_α must also appear in top-$k(S_\alpha)$: fewer than k elements of $S_{\alpha_{\max}}$ precede it, hence fewer than k elements of the subset S_α precede it. Therefore $R_\alpha \cap R_{\max} = S_\alpha \cap R_{\max}$. Because the survivor sets are nested, this intersection can only grow with α, so $D(\alpha)$ is non-increasing. □

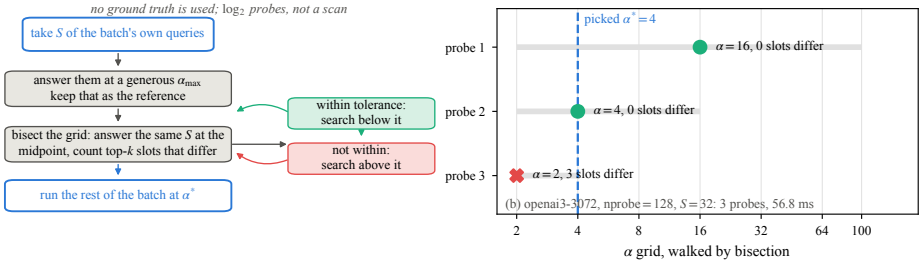

Fig. 4: Label-free output-stability calibration. Ground truth is not used during α selection.

Implementation caveat and measured deviation. Proposition 1 is exact for the ideal top-c_k selector, but the production GPU scan uses a bounded per-thread selector with `K_LOCAL`=4. A thread can therefore discard a globally top-c_k item if more than `K_LOCAL` such items fall in its stride class, so theorem-level nestedness is not asserted for the deployed kernel. We quantify this approximation directly rather than hiding it behind aggregate recall: with the index pinned, vogue-768 at *nprobe*=128, and `BLOCK`=512, increasing `K_LOCAL` from 4 to 8 changes 11 of 10,000 returned ID positions, touches 4 of 1,000 queries, and gives 1 query a genuinely different result set. `BLOCK`=512 is deliberately the harder collision regime; `K_LOCAL`=8 is register-infeasible at the production `BLOCK`=1024. Thus the departure from exact selection is small but nonzero in this stress test.

Because nestedness is not guaranteed, the soundness of bisection cannot rest on Proposition 1 for the deployed system, and we do not claim it does. We instead check the predicate the search actually relies on. Across all 15 measured calibration traces that probe two or more grid points, the sampled disagreement $D(\alpha)$ is non-increasing in α without exception. Within this evidence bisection returns the smallest grid α its sampled criterion accepts; outside it, the stopping behaviour is conservative relative to that criterion rather than certified. Floating-point ties are a separate implementation issue: the proposition assumes a deterministic total order, not bitwise-identical floating-point association.

Sampled parameter tuning itself is not new. FAISS [5] exposes parameter-space exploration, and Li et al. [6] learn adaptive ANN termination. Our contribution is narrower: a predictor-free top-k stability signal applied to JHQ's external residual budget, together with explicit validation of proxy error and calibration economics. The method trades per-query adaptivity for a simple workload-level control loop that can be amortised.

4.4 Calibration cost and reuse

Calibration is only useful if its one-time cost is amortised. Let T_{cal} be calibration time, T_{fixed} the per-batch time with a fixed baseline α, and T_{rule} the per-batch time after selection. For B subsequent batches,

$$T_{\text{adaptive}}(B) = T_{\text{cal}} + B\,T_{\text{rule}}, \qquad T_{\text{fixed}}^{\text{total}}(B) = B\,T_{\text{fixed}}, \tag{4}$$

and, when $T_{\text{rule}} < T_{\text{fixed}}$, the continuous break-even point is $B^* = T_{\text{cal}}/(T_{\text{fixed}} - T_{\text{rule}})$. This formula makes the policy trade-off explicit: a more expensive calibration is rational only when the selected budget saves enough steady-state work and the operating point persists long enough. Across 32 configurations where the rule changes the budget, steady-state gain is 1.044–2.653× and payback is 0.5–21.0 batches, with median 1.75. Large payback near gain=1 is partly definitional through the denominator, so we do not present the correlation as a new systems mechanism.

Reuse assumes the query distribution, routing configuration, k, and index remain sufficiently stable. This is the price of workload-level rather than per-query adaptation. Distribution drift is not yet measured. If recalibration occurs every H batches, a simple accounting adds T_{cal}/H per batch, but selecting H itself becomes a drift-detection problem outside the current evidence. Our calibration timer begins after sample copying, so the reported cost is algorithm-level rather than a complete production control-plane cost.

5 Related Work

We organise related work around the novelty threats that matter for this paper rather than as a catalogue.

Quantisation and JHQ. Product Quantization (PQ) [4] established Cartesian decomposition across subspaces and asymmetric distance computation. JHQ [3] targets high-dimensional ANN with an orthogonal transform, a Cartesian-constructed primary quantiser, and hierarchical residual refinement. We do not claim JHQ's hierarchy or orthogonal transform as contributions; our work begins from that representation and asks how its specific structure should execute on a GPU.

Small-table distance evaluation. PQ Fast Scan [1] showed that cache residency alone is insufficient for fast ADC and reorganised PQ scanning around small tables and SIMD-register lookups. Quicker ADC [2] generalised this direction with irregular product quantisers and split tables for wider code widths. These works make it essential not to claim novelty merely from "smaller LUTs." The distinction is in provenance, not in loss: Quicker ADC obtains small tables by changing the quantiser, replacing 8-bit sub-quantisers with narrower irregular ones whose tables are small by construction. Our table is small because JHQ's existing 8-bit Cartesian subspace code already separates exactly (Equation 1), so we preserve the represented codebook and derive an identity for its distance table rather than choosing a different one. We also show empirically that making the table even smaller can be slower because extra lookup instructions dominate — most sharply between G=4 and G=8, whose tables are byte-identical. Thus our claim is about exploiting a particular representation, not about inventing small-table ADC in general.

Adaptive ANN control. ANN libraries such as FAISS [5] commonly expose parameter sweeps and tuning utilities, while learned approaches can adapt search termination to the query [6]. We therefore do not claim sampling or parameter selection as new. Our output-stability rule differs in its signal and deployment assumptions: it needs no labelled nearest neighbours and no learned predictor, compares the system's own returned top-k sets to a generous-budget reference, and chooses the externally specified JHQ refinement budget. The evaluation separately quantifies selection error and amortisation, which are necessary because a label-free proxy is not a recall guarantee.

GPU ANN systems. CAGRA [7] represents the graph-based GPU frontier, and cuVS provides production implementations of CAGRA, IVF-PQ, and IVF-RaBitQ. GPU IVF-RaBitQ [8] combines inverted-file routing with low-bit RaBitQ codes and reports a strong recall–throughput/build/storage trade-off. These systems are not uniformly weaker than JHQ: our matched experiments show regime crossings in the high-recall tail and with batch size. This is why we report nondominated frontiers and configuration-specific build failures rather than a single aggregate speedup.

6 Experiments

6.1 Experimental setup and protocol

We evaluate on one NVIDIA RTX 5090. The frozen run records 32,607 MiB device memory, 170 SMs, a 101,376-byte opt-in shared-memory limit per block, and 1,792 GB/s nominal device bandwidth; the host has 208 logical cores and 754 GB RAM. All compared GPU systems in our measurements use this card. The six datasets are vogue-768 (about 1.0M$\times$768), arxiv-768 (2.25M$\times$768), bge-m3 (10.1M $\times$ 1024), stella-trec24 (17.8M $\times$ 1024), openai3-1536 (1.0M $\times$ 1536), and openai3-3072 (1.0M $\times$ 3072). When we refer to attainable occupancy, it is derived from per-block shared-memory budgets and resident-block limits, not from hardware performance counters: Nsight Compute counters were unavailable in the benchmark container (`ERR_NVGPUCTRPERM`). We therefore do not report profiler-measured occupancy. Where a configuration was measured more than once — repetition counts range from one to three across the granularity grid — every ratio we quote is the median of its repeats, so a single outlying run cannot set an interval endpoint.

We report Recall@10 against true top-10 ground truth and use k=10 throughout the current evaluation. Because the integer survivor budget c_k scales with αk, generalisation to other k is not yet established. The common timing boundary is host query input to host result output. Recall–QPS frontiers use steady-state throughput at a batch of 1,024, which is the whole query set in one call; Section 6.6 shows this choice matters and reports the batch sweep separately. Calibration time is reported separately and incorporated through break-even analysis. Diagnostic α-sweep QPS values that include readback instrumentation are not used in the frontier.

The main GPU baselines are IVF-RaBitQ, CAGRA (fp32 or int8 where necessary), and IVF-PQ. The reported JHQ runs use dense coarse training with `JHQ_N_TRAIN` $= 39 \cdot nlist$. We do not mix earlier under-trained or pre-fix points into the final frontier, and we do not union fronts measured by different binaries. A scan endpoint is a measured endpoint, not an architectural recall ceiling. Cross-build recall variability is about 10^{-3} in the documented cold-start experiment, while paired comparisons on the same index show a lower approximately 10^{-4} floor from top-c_k tie breaking. Those two floors serve different purposes and are not interchangeable; in particular, Pareto fronts are built after bucketing recall at 10^{-3}, so a tie-break-sized difference cannot buy a point a place on a front. All figures are taken directly from the frozen project plotting outputs; we do not redraw benchmark curves or reconstruct values from prose summaries.

The CPU JHQ baseline runs the same index parameters on the same host with threads pinned (`OMP_PROC_BIND=close`, `OMP_PLACES=cores`) and its own $(\alpha, nprobe)$ sweep, so both sides are compared on their own envelopes rather than one side's envelope against the other's worst point. It shares the training procedure and parameters with the GPU index but not a codebook, and each cell is measured once; Section 6.7 states the resulting precision.

6.2 RQ1: end-to-end recall–throughput frontier

Figure 5 shows the measured Recall@10–QPS frontiers for JHQ and the available GPU baselines across all six datasets, at batch 1,024. Four datasets have complete JHQ/IVF-RaBitQ comparisons; bge-m3 and stella-trec24 show the baselines that could be constructed and searched on the tested card. We use the measured curves as the primary evidence and quote matched-recall ratios only as summaries. On the four directly comparable datasets, JHQ leads through Recall@10=0.95 at this batch. At d=768 the margin is close to parity by 0.95 and reverses at higher recall. At d=1536, openai3-1536 remains 1.36–2.35× ahead across the reported 0.90–0.98 range. Openai3-3072 is 1.82× at 0.90, 1.36× at 0.95, 1.06× at 0.97, and 0.93× at 0.98. These ratios are batch-conditional: Section 6.6 shows the same openai3-3072 comparison ranging from 1.34× to 2.14× at Recall@10=0.95 as batch varies, and vogue-768 falling below parity at small batch. These observations define an operating region; they do not establish dimension alone as a causal explanation.

On bge-m3 and stella-trec24, the tested cuVS IVF-RaBitQ host-input build path fails with an out-of-memory allocation after entering its streaming construction path, and CAGRA fp32 does not fit the card. JHQ indexes both in the measured configuration. We phrase this narrowly: the measured library/configuration paths do not produce a comparable index on this device. It would be incorrect to claim that the underlying algorithms can never index those datasets. CAGRA int8 and IVF-PQ remain useful comparison points where their measured recall ranges overlap.

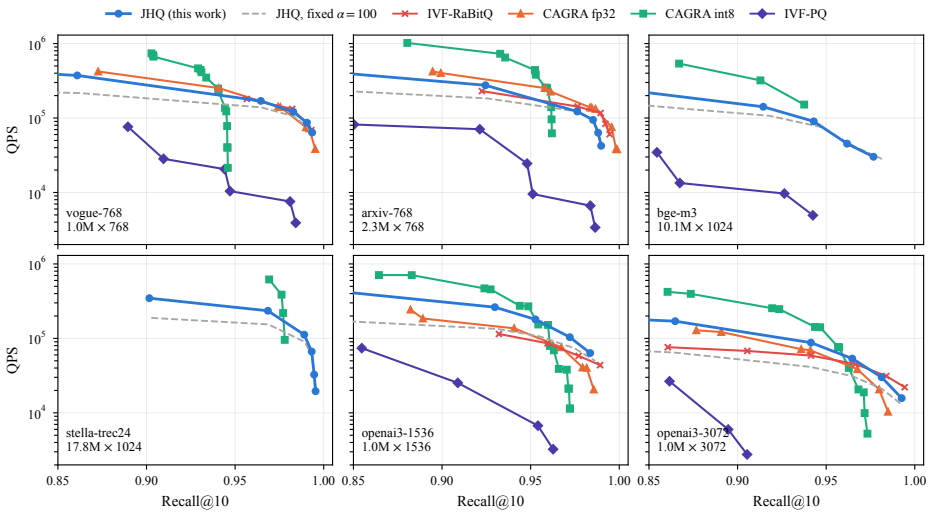

Fig. 5: Measured Recall@10–QPS frontiers across all six datasets, at batch 1,024. Four datasets include complete JHQ/IVF-RaBitQ curves; the two larger datasets show the baselines available on the tested GPU. Sweep endpoints are measured endpoints, not architectural recall ceilings.

6.3 RQ2: adaptive refinement

At *nprobe*=128, the output-stability rule recovers the α selected by the diagnostic sweep in three of four evaluated configurations. The remaining arxiv configuration is censored by $\alpha_{\max}$: its sweep is still improving at α=200 while the rule cannot select beyond its own upper bound. This is a limitation of the chosen grid, not evidence that the rule discovered the true saturation point.

Sampling risk is workload dependent. In 2,000 held-out resampled draws, S=32 keeps held-out loss within 10^{-3} in 76% of draws for openai3-3072 but only 27% for vogue-768. Vogue's mean held-out loss at S=32 is 0.0033 and its 95th percentile is 0.0110. Openai3-3072 reaches 98% within 10^{-3} at S=64, whereas vogue still has 11% of draws outside the threshold at S=128. Therefore S=32 is not a universal knee. A production policy should choose sample size against an explicit risk target for each workload.

Tolerance also matters. With the current grid, zero disagreeing slots leaves about 26% of attainable throughput on the table, whereas allowing two slots incurs about 0.0035 recall loss in the reported experiment. We use ϵ=1 as an empirical compromise, not a guarantee. The *nprobe*=512 full-sweep validation remains unavailable and is not inferred from *nprobe*=128.

6.4 RQ3: do the design trade-offs behave as predicted?

We test the alternatives that Section 3 rejected, not only the final implementation. Figure 7 first varies the factorisation granularity G while holding the represented distance fixed. If table size alone governed performance, G=4 or G=8

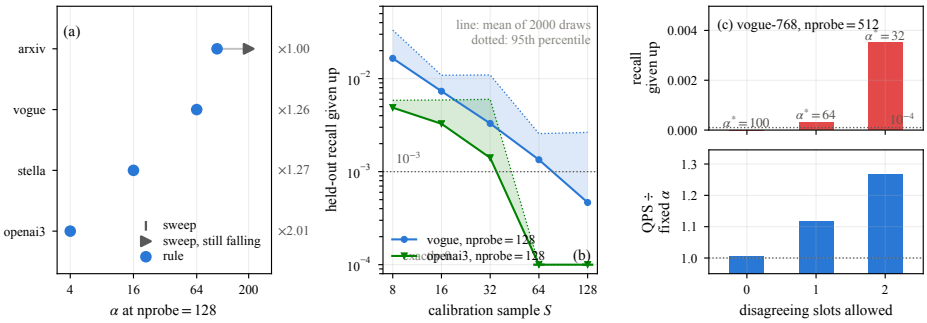

Fig. 6: Calibration validation. (a) Selected α versus the exhaustive sweep where a same-operating-point sweep exists, annotated with the throughput the selection gains; (b) held-out resampling shows that sample-size risk is workload dependent; (c) tolerance changes the selected budget and quality–cost trade-off.

should dominate G=2 because their tables are smaller. They do not: G=2 is never beaten across the eight configurations, while throughput falls as lookups per candidate rise. The sharpest form of that result is internal to the losers. $G \cdot 2^{8/G}$ is 16 for both G=4 and G=8, so those two tables are byte-identical and the only difference between them is four versus eight lookups per candidate–subspace; the eight-way form is 5.8% slower at *nprobe*=8 and 38.3% slower at *nprobe*=512. Residency is held exactly constant and issued work doubles, which is the cleanest separation of the two effects in this paper.

The same figure shows the expected regime change with M. Against the full table, factorisation is worth 0–11% at M=96 and 8–53% at M=384, and in both cases the gain rises monotonically with probe depth: on openai3-3072 it is +8.3% at *nprobe*=8, +9.8% at 32, +22.6% at 128 and +53.4% at 512. We report the trend rather than a flat interval because the trend is the mechanism: a non-resident table is paid once per candidate per subspace, so its cost grows both with M and with the number of candidates scanned. This is evidence for a residency budget followed by an instruction-cost trade-off, not for a monotonic "smaller LUT is faster" rule.

Two further controls separate bytes from issued work. Packed loading moves the same logical primary-code bytes but gains 30–48%, while carrying a private probe cursor removes redundant inner-loop work and yields +2.5% to +148% across measured settings. The latter is the largest effect size but an implementation correction, not a research contribution. Its value is diagnostic: together with the packing result, it predicts that reductions in issued instructions can matter even when memory volume is unchanged. Adaptive α is excluded from this ablation because it changes the amount of algorithmic work and can change recall.

6.5 RQ4: can plausible alternative explanations be rejected?

Figure 8 provides two negative controls. First, the table-free exact primary distance removes most LUT state and, on openai3-3072, raises the attainable

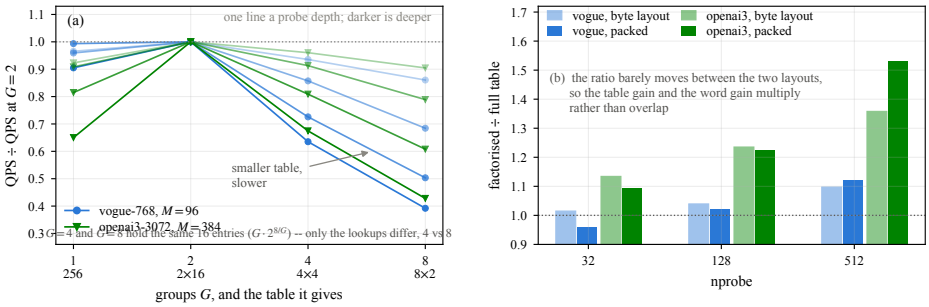

Fig. 7: Why the exact LUT is split into two groups. (a) G=2 is never beaten, although G=4 and G=8 use smaller tables and are byte-identical to each other, exposing the lookup-instruction trade-off; (b) factorisation matters far more at M=384 than M=96 and remains complementary to packed loading. Panel (b) is the two-by-two layout phase, measured at $nprobe \in \{32, 128, 512\}$; the $nprobe$=8 point quoted in the text is the lightest probe-depth line of panel (a).

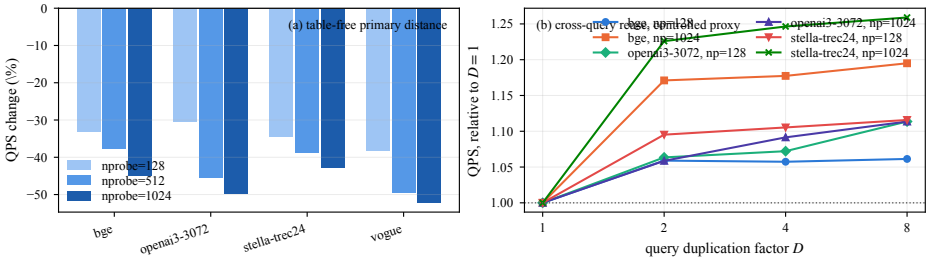

Fig. 8: Negative controls for mechanism. (a) Table-free exact primary distance reduces LUT state yet loses 30–52%; (b) controlled query duplication isolates cross-query reuse and shows only modest additional throughput. These reject simplistic "fewer bytes is always faster" and "reuse alone will solve the scan" explanations.

resident-block count implied by the shared-memory budget; nevertheless it is 30–52% slower. This rejects the simple explanation that table residency or attainable occupancy alone is the bottleneck. These occupancy statements are analytical resource-budget calculations as specified in Section 6.1, not Nsight measurements. Second, duplicating queries creates an intentionally strong cross-query-reuse proxy. Throughput improves only 4–26% and the gain largely saturates by duplication factor two, consistent with much of the reusable working set already being absorbed by the 96 MB L2 cache. This does not prove that a cluster-centric scheduler cannot help — such a rewrite would also change scheduling and table amortisation — but it shows that the reuse term by itself is modest.

The remaining negatives point in the same direction. G=4 and G=8 reduce table entries further but lose to extra lookups. In the frozen negative-control records for stella-trec24, fusing residual refinement is 14.2% slower than the 68,565-QPS baseline at $nprobe$=128 and 14.8% slower than the 16,126-QPS base-

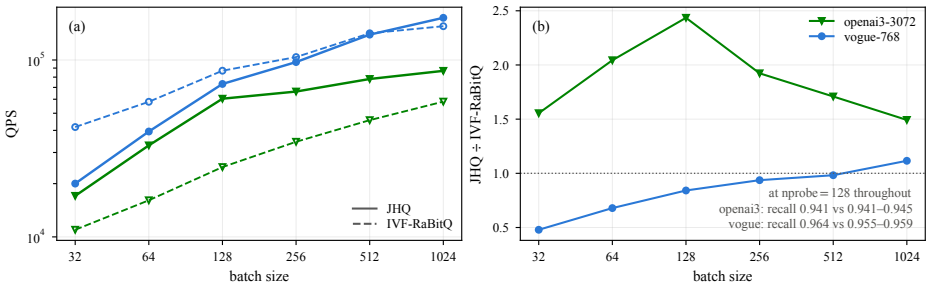

Fig. 9: Equal-recall batch-size sensitivity. Curves show JHQ/IVF-RaBitQ throughput at matched Recall@10=0.90 and 0.95. JHQ leads at every shown openai3-3072 point, while vogue-768 crosses parity between batch 128 and 512 at both recall targets. The excluded openai3-3072 batch-1,024, R=0.97 cell is not shown.

line at *nprobe*=512; the removed intermediate round trip is outweighed by the combined shared-memory/register footprint and longer fused-kernel residency. LUT construction occupies only 0.2–2.3% of a batch, ruling out faster table building as the main explanation for factorisation. Taken together with packing and cursor controls, the evidence is consistent with an issue-sensitive primary scan on this RTX 5090. We deliberately say "consistent with" rather than claim a universal GPU law: the experiments isolate several competing explanations on one architecture, but cross-GPU validation remains open, and without profiler counters we infer issue sensitivity from controlled comparisons rather than measure it directly.

6.6 RQ5: batch-size sensitivity at equal recall

Figure 9 replaces a fixed-nprobe comparison with matched-recall measurements at Recall@10=0.90 and 0.95. On openai3-3072, JHQ is faster at every measured batch and both recall targets: for batches 32, 128, 512, and 1,024, the JHQ/IVF-RaBitQ ratios are 2.23/1.34, 3.19/2.14, 2.30/1.51, and 1.89/1.41 at R=0.90/R=0.95. On vogue-768, JHQ begins below parity but crosses between batch 128 and 512: the corresponding ratios are 0.68/0.55, 0.99/0.92, 1.06/1.01, and 1.09/1.33. Thus batch size can change the system ordering, and the frontier ratios of Section 6.2 should be read as the batch-1,024 column of this sweep rather than as batch-independent summaries. The openai3-3072 batch-1,024, R=0.97 cell is deliberately not used because its RaBitQ anchor is the one matched point that failed the project's monotonicity check. We also do not manufacture a 10k-query point by duplicating the roughly 1k available queries, because the reuse control in Figure 8 shows that repetition itself inflates throughput.

6.7 RQ6: what the adaptive budget is worth, CPU against GPU

Figure 10(a) places both devices' $(\alpha, nprobe)$ envelopes on one recall axis. At matched recall the GPU rule arm reaches 52× the CPU envelope at Recall@10=0.9645

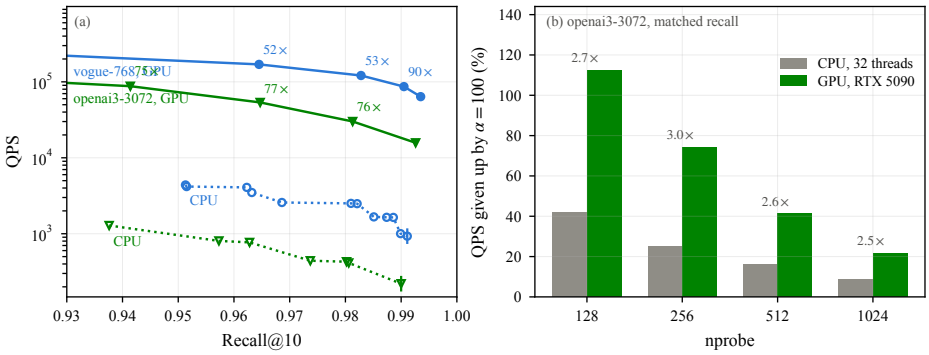

Fig. 10: CPU baseline. (a) Both devices' (α, *nprobe*) envelopes, with the matched-recall ratio annotated only where both cover the recall; the tick marks where the CPU envelope ends. (b) The cost of pinning α=100 at matched recall is 2.5–3× larger on the GPU than on the CPU at every probe depth.

on vogue-768 and 54× at 0.9828; on openai3-3072 it is 75–77× across 0.94–0.98. With α pinned to 100 on both sides the same comparison gives 43–48× and 35–54×. Above Recall@10=0.9911 on vogue-768 and 0.9900 on openai3-3072 the CPU envelope has no points, and we report those GPU operating points as outside its range rather than dividing by the CPU's last measurement.

Two properties of this measurement bound how it should be read. Each cell is measured once and cell-level throughput noise reaches roughly 2× — the same α=2 configuration reads 9,015 QPS at 16 threads and 4,090 at 32 — so these are order-of-magnitude figures. The bias direction, however, is known: the envelope keeps the fastest point in each recall bucket, so noise raises the CPU side and lowers the ratio. The reported range is a lower bound with respect to it. We also drop CPU rows below *nprobe*=128, where 1,000 queries complete fast enough that thread start-up dominates the timer and measured throughput rises with *nprobe*; the recall column is unaffected and the throughput column is not usable there.

The comparison worth making is not the raw ratio. Figure 10(b) asks the same question of both devices: how much throughput does pinning α=100 give up, at equal recall, on the same dataset and nprobe grid? On the CPU it costs 41.8%, 25.1%, 16.0% and 8.7% at $nprobe = 128, 256, 512, 1024$; on the GPU it costs 112.5%, 74.1%, 41.3% and 21.8% — between 2.5 and 3.0× more at every point. The mechanism is the stage decomposition already measured: the GPU scan is fast enough that residual refinement occupies a larger share of the total, so the same reduction in survivors buys more. Selective refinement is therefore not a CPU idea carried across unchanged; the control JHQ leaves externally specified is worth several times more on the device we port it to, which is why Section 4 spends a calibration pass on choosing it.

6.8 RQ7: construction and memory

Across the tested datasets, observed build-time ranges are 1.1–14.3 s for JHQ, 9–75 s for CAGRA int8, 6.4–43 s for IVF-PQ, and 4.5–14.0 s for IVF-RaBitQ where that path builds. These ranges describe tested configurations rather than intrinsic algorithmic bounds. Build accounting must also respect caching: training can be cached while add is not, so independently taking maxima and summing them can fabricate a time that never occurred in one run.

JHQ's logical code budget in the paper setting is about 9 bits per dimension before routing and implementation metadata. In the four datasets with measured IVF-RaBitQ VRAM, the current evidence reports IVF-RaBitQ using 22–39% less device memory than JHQ. We therefore do not claim a universal memory advantage. JHQ instead occupies a different memory–accuracy–throughput operating region and, in our tested paths, still indexes the two largest datasets where the compared RaBitQ/CAGRA paths fail or do not fit. Memory accounting also contains an "other/unaccounted" component; it should not be relabelled as allocator or context overhead without a direct measurement.

Overall, the experiments support the paper's bounded thesis. Exact representation-aware changes matter most when they alter the hardware regime; output-stability calibration can remove unnecessary residual work, but its sample size and amortisation are workload dependent; and the resulting system is competitive over a substantial but not universal region of the ANN frontier.

7 Conclusion

We studied JHQ as a representation-to-hardware mapping problem rather than a sequence of CUDA tricks. The design follows from explicit alternatives. Candidate-parallel scan determines the physical layout; $D_s{=}B{=}8$ exposes a natural packed word that reduces issued work without pretending to reduce bytes; and the Cartesian primary code creates a computation–residency trade-off between full LUTs, symbolic evaluation, and exact factorisation. This is an M-scaling property, not a one-off effect: full-table state grows as $256M$ entries while the two-way factorised state grows as $32M$, so the factorised table carries an $8\times$ smaller coefficient against a fixed on-chip budget. On the tested card that budget is 59–94 KiB once the scan's own candidate scratch is deducted, which no full table in this paper meets and every factorised table clears; the cost of a table that misses is then paid once per candidate per subspace, so it grows with M and with probe depth together. Consistent with that prediction, the measured factorisation effect is 0–11% at $M{=}96$ and 8–53% at $M{=}384$, rising monotonically with probe depth in both. The measured optimum is not the smallest representation: two 16-entry tables outperform both the full table and still-smaller four- and eight-way splits, and the clearest evidence is that the four- and eight-way tables are byte-identical to each other yet differ by up to 38% in throughput. Negative controls therefore explain why the chosen design works instead of merely showing that it wins.

We also preserve the hierarchy's other systems advantage: expensive per-dimension residual data are read only after primary screening. Because the use-

ful survivor count varies sharply by workload, a label-free output-stability rule chooses the refinement factor from sampled workload queries. A matched CPU baseline shows this is a device-specific opportunity rather than an inherited one: the same α knob is worth 2.5–3$\times$ more on the GPU than on the CPU at every probe depth, because a faster scan leaves refinement a larger share of the total. The rule's limitations remain explicit: $\alpha_{\max}$ bounds what it can discover, sampling error is workload dependent, amortisation assumes a stable operating regime, and its bisection is justified by 15 calibration traces in which sampled disagreement was monotone rather than by a theorem covering the bounded selector the kernel actually runs.

Together, these contributions define a bounded operating region rather than a universal winner. JHQ is competitive through moderate-to-high recall on the directly comparable datasets at large batch, but other GPU ANN methods win in parts of the high-recall and batch-size space, and IVF-RaBitQ can use less device memory. The broader lesson is methodological: representation-specific GPU optimisation should be justified by the resource trade-off it changes and by alternatives that fail for identifiable reasons. Remaining evidence gaps are k sensitivity beyond 10, workload-drift and recalibration behaviour, direct instruction-count measurement in place of the controlled comparisons that currently stand in for it, and cross-GPU validation.

References

1. André, F., Kermarrec, A.M., Scouarnec, N.L.: Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan. Proc. VLDB Endow. **9**(4), 288–299 (2015). https://doi.org/10.14778/2856318.2856324
2. André, F., Kermarrec, A.M., Scouarnec, N.L.: Quicker ADC: Unlocking the hidden potential of product quantization with SIMD. IEEE Trans. Pattern Anal. Mach. Intell. **43**(5), 1666–1677 (2021). https://doi.org/10.1109/TPAMI.2019.2952606
3. Han, J., Zhang, M., Trajcevski, G.: JHQ: Johnson-Lindenstrauss enhanced hierarchical quantization for high-dimensional approximate nearest neighbor search. Proc. VLDB Endow. **19**(7), 1530–1543 (2026). https://doi.org/10.14778/3801059.3801067
4. Jégou, H., Douze, M., Schmid, C.: Product quantization for nearest neighbor search. IEEE Trans. Pattern Anal. Mach. Intell. **33**(1), 117–128 (2011). https://doi.org/10.1109/TPAMI.2010.57
5. Johnson, J., Douze, M., Jégou, H.: Billion-scale similarity search with GPUs. arXiv:1702.08734 (2017)
6. Li, C., Zhang, M., Andersen, D.G., He, Y.: Improving approximate nearest neighbor search through learned adaptive early termination. In: SIGMOD. pp. 2539–2554 (2020). https://doi.org/10.1145/3318464.3380600
7. Ootomo, H., Naruse, A., Nolet, C., Wang, R., Feher, T., Wang, Y.: CAGRA: Highly parallel graph construction and approximate nearest neighbor search for GPUs. In: IEEE ICDE (2024). https://doi.org/10.1109/ICDE60146.2024.00323
8. Shi, J., Gao, J., Xia, J., Feher, T.B., Long, C.: GPU-native approximate nearest neighbor search with IVF-RaBitQ: Fast index build and search. Proc. VLDB Endow. **19**(11), 3454–3467 (2026). https://doi.org/10.14778/3836663.3836701